CS3242 3D Modeling and Animation

Assignment: Modeling

In this project, first, we will build the subset of the functionalities. Second, we will build the `fnext` operation.

OBJ Wavefront Files

For the assignment, we will create a mesh by reading in OBJ files. OBJ files are first created by Wavefront technology and adopted by many other 3D software as a file standard. The file type stores a lot of information such as geometry and texture, etc. However, we will only use the vertex and triangle information for this assignment.

Vertices are stored with a line beginning with the character "`v`" and followed by the three numbers for its x, y and z coordinates. (But "`vt`" and "`vn`" are storing other information, such as vertex normal vectors.) Basically, each vertex is given an index according to its order of appearance in the file, starting with 1 (instead of 0). So, the indices of the vertices will be ranging from 1 to n, i.e. the number of vertices.

Faces are stored with a line beginning with the character "`f`" and followed by some integers. The simplest form is an "`f`" followed by three integers, in which, representing the three vertices by the vertex indices. OBJ files allow a face with more than 3 vertices, but we will use those files with *triangles only* in this assignment.

If you would like to understand more about OBJ files, you can google "`obj wavefront file`" or visit some website such as http://www.fileformat.info/format/wavefrontobj/egff.htm.

Skeleton Code

No matter you are using Windows or Mac, there are three c++ files for you to work on, namely, main.cpp and mesh.{h, cpp}. You can download the project files, or only the .{h, cpp} files and create your own project.

The `main.cpp` provides you with some basic visualization for some 3D objects and allows some keyboard I/O. Basically, you should not need to change any code in it, except if you need to add more testing code or new functionalities of your own.

The mesh.{h, cpp} are the files you will use for this half of the semester. Let's see what the header file is first. For simplicity and efficiency, we put all the code for an **OBJ file** and the **Trist** structure together. Namely, we did not separate the code or write two classes for them.

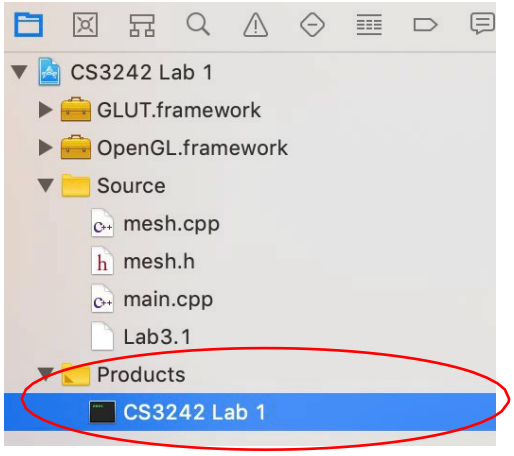
```
class myObjType {
    int vcount = 0;
    int tcount = 0;
    double vlist[MAXV][3];   // vertices list
    int tlist[MAXT][3];      // triangle list
    int fnlist[MAXT][3];     // fnext list for future (not this assignment)
    double nlist[MAXT][3];   // storing triangle normals
    double lmax[3];          // the maximum coordinates of x,y,z
    double lmin[3];          // the minimum coordinates of x,y,z

    int statMinAngle[18]; // each bucket is  degrees has a 10 degree range from 0 to 180 degree
    int statMaxAngle[18];
public:
    myObjType() { vcount = 0; tcount = 0; };
    void readFile(char* filename);  // assumming file contains a manifold
    void writeFile(char* filename);
    void draw();
    void computeStat();
};
```

Member functions

The member function `readfile()` will read in an .obj file and store all the vertices into `vlist` and the vertex indices of a triangle into `flist`. And the numbers of vertices and triangles are stored in the variables `vcount` and `tcount` respectively. In the process of reading, `lmin[3]` will store the minimums of x, y, z values of all the vertices, and `lmax[3]` for the maximums. Following the conventions for the vertices, the indices for the triangles will range from 1 to `tcount`, in which, `flist[0]` is not used.

Note that main.cpp will ask you to input a filename in the console. Some may have trouble to locate the files because you do not know which directory your executable running is in.

For **Mac** users, you can click the "Products" folder in your XCode project. And on the right, you will find the "Full Path". After clicking the little right grey arrow on the right, the location/folder of the executable will be opened. And you can copy your .obj file to this folder to be opened by your program.



For **Windows** users, you may copy the file into your executable folder like Mac users. However, I found it not so ideal because those .obj files should not be a part of the code/project. My suggestion is to place the .obj files at the same folder as your project file. So if you want to open a file, e.g. `cat.obj`, you can type "`../../cat.obj`" to open it.

Lab 01 version 1.0 skeleton	3/10/2019 6:11 PM	File folder	
Lab 01 version 1.0 solution	3/10/2019 6:11 PM	File folder	
Mac version skeleton	3/9/2019 5:40 PM	File folder	
Mac version solution	3/10/2019 6:12 PM	File folder	
Alucy.obj	1/6/2017 11:16 PM	OBJ File	32,503 KB
cat.obj	2/25/2017 6:27 PM	OBJ File	204 KB
cat.out.obj	3/9/2019 10:53 AM	OBJ File	83 KB
cow.obj	3/8/2019 4:46 PM	OBJ File	228 KB
cs3242 lab 1.docx	3/11/2019 8:19 AM	Microsoft Word D...	1,316 KB
deer.obj	2/25/2017 7:26 PM	OBJ File	158 KB
destoryer.obj	3/1/2004 5:01 PM	OBJ File	105,267 KB
Lab 01 version 1.0 skeleton.zip	3/11/2019 8:02 AM	Compressed (zipp...	110,245 KB
Mac version skeleton.zip	3/11/2019 8:03 AM	Compressed (zipp...	55 KB
pumpkin.obj	3/8/2019 4:46 PM	OBJ File	326 KB
Skull.obj	3/8/2019 7:32 PM	OBJ File	2,479 KB
smallcase.obj	3/8/2019 9:28 PM	OBJ File	1 KB
smallcaseflipped.obj	3/8/2019 9:28 PM	OBJ File	1 KB
teapot.obj	3/8/2019 4:45 PM	OBJ File	206 KB
teddy.obj	3/8/2019 4:46 PM	OBJ File	90 KB
trex.obj	9/18/2012 7:48 PM	OBJ File	1,318 KB

At the end of `readfile()`, the function `computeStat()` will be called. The function `computeStat()` will compute some statistics of the mesh, such as angles of the triangles, but it is empty now because you are supposed to implement them as one of the tasks.

The function `draw()` draws all the stored triangles by OpenGL. However, it will only draw the mesh without shading effect. If you press "W", you will see the wireframe of your file. However, every lighting environment is already set up for you in the code, except the normal vectors of the triangles of the mesh. You will compute them as one of the tasks, as well as the function `writefile()`.

Tasks

1. Computing Normal Vectors

Compute all normal vectors of each triangle and store them into `nlist` with the same index. Uncomment the code "`glNormal3dv`" in the `draw()` function so that your mesh will be drawn with correct shading. The normal computations should be done in the function `readfile()`.

2. Compute Angle Statistics

For each triangle, it has a maximum angle and a minimum angle. This task is to record the frequencies of them into the two arrays `statMaxAngle` and `statMinAngle`. Each array has 18 buckets for angles from 0 to 180 degrees, i.e. each bucket has a 10-degree range. For example, if there are three triangles with minimum angles 9, 3 and 55, then `statMinAngle[0] = 2` and `statMinAngle[5] = 1`. The statistics will be printed at the end of `computeStat()`. Just write the current mesh into another obj file. If you press "O", the file main.cpp will call your `writefile()`. You must produce the "`v`" and "`f`" lines. You should be able to read back your file with `readfile()`.

3. Implement functions enext(), sym(), org(), dest()

As mentioned in the lecture notes, they could be separate functions outside of the class.

4. Implement fnext()

After reading an OBJ file, link up all the triangles by setting up the `fnlist`. To prove your

implementation is right, you need to implement the following to show that your code works.

5. Compute the Number of Components

An OBJ file may contain a few pieces of surfaces, and the number of pieces is the same as the definition of the number of components in a graph. You may simply achieve this by BFS or DFS like a graph. However, by being in the same component we only mean that their triangles are connected by `fnlist`. Namely, two triangles can be in two different components if they intersect each other, but they are not connected through some vertices and edges.

6. Compute Vertex Normal Vectors for Smooth Shading

OpenGL can render the Gouraud and Phong shadings by averaged vertex normal vectors. Create a new vector array inside your class and name it `vnlist`. It stores all the vertex normal vectors that is the average of those of surrounding triangles. And add a keyboard function in your `main.cpp` so that you can choose to toggle flat or smooth shadings.

7. Visualize boundary edges

Color the edges shared by only exactly one triangle by red.

The deadline: 10 Apr, Fri, 11:59pm, submit the project file whose file name is your name_assignment2.zip without .obj files.